

```
[] board.py
>[] game.py
>[] main.py
[text/markdown] README.md
>[] rules.py
Read README.md fully and understand all the python files, then
identify all the bugs
```



I've read everything in full. I'm saving my reference notes to a `.md` file first, and cross-checking the checkers rules I'll rely on.

I read all five files in full and saved my notes to `checkers_notes.md`, which I'll use as my reference from here on. I haven't run anything, so everything below comes from reading the code. I haven't written any code, per your preference.

Assumptions I'm making

1. The target is American checkers on an 8×8 board, since the code already uses 12 pieces per side, one-square kings and short jumps. In that game, men can jump only forwards and kings move and capture in both directions, but only one square at a time. I cross-checked this against a few rules sources, and they agree.
2. The README is your spec. Where it is silent, I use the standard rules.

Bugs in the current code (Tasks 1–4)

Task 1: capture correctness

1. **Captured pieces are never removed.** `move_piece` in `board.py` only moves the mover, and nothing clears the jumped square. The victim stays on the board and can be jumped again, which is exactly the defect the README describes.
2. **A man can jump its own king.** In `rules.py`, `capture_move` checks `board[mr][mc] not in (".", player)`. Your own king is stored as `"RK"` or `"BK"`, which isn't equal to `player`, so it passes as an enemy piece.

Task 2: win and no-move detection 3. **The game never ends.** `run()` in `game.py` loops forever and only exits on `q`. It doesn't check for a side with no pieces or no legal moves. The rules sources agree that you win by capturing all of your opponent's pieces or blocking them so they have no legal move.

4. **A design gap, not a bug.** `rules.py` can only check one specific move, with no function that lists a player's legal moves. Tasks 2 and 3 both need that, so it will have to exist.

Task 3: complete rules 5. **Kings can't move or capture backwards.** `simple_move` and `capture_move` use `direction = -1` if `player == "R"` else `1` and ignore whether the piece is a king. `game.py` accepts `"RK"` as your piece, so a king just behaves like a man. 6. **No forced capture.** `game.py` accepts any legal simple move even when a capture is available. The rule is that if any of your pieces can jump, you are not allowed to play anything else. 7. **No multi-capture.** After one jump, `run()` immediately switches the player. In the real rules, if another jump is available from the landing square, you must continue jumping in the same turn. 8. **Promotion timing needs care.** `promote()` scans the whole board after every move. That works today because there is no multi-jump, but once you add it, a man that reaches the back row mid-chain must end its turn. The rules say that crowning ends your turn, even if the brand-new king could jump again. 9. **A jumped piece can't be jumped twice in one chain.** Bug 1 already breaks this, and it needs handling again once multi-jump exists.

Task 4: feedback 10. **There is no feedback at all.** Successful moves, captures and promotions print nothing. The only messages are for errors, and a rejected move always says the vague "Invalid move."

Other bugs and problems I found

11. **The board display misaligns after promotion.** Kings are stored as two characters (`"RK"`), while everything else is one character. `print_board` joins with a single space, so any row containing a king shifts out of its column.
12. **Unhandled input exits.** Ctrl-D (EOF) or Ctrl-C at the prompt gives a traceback, not a clean quit. Only exactly `q` works as quit, and `quit` gives "Enter four coordinates."
13. **SIZE is defined twice**, in `board.py` and `rules.py` . It works today but can drift.
14. **Rules functions don't check bounds.** They rely on `game.py` having validated coordinates first. That is fine now, but fragile once you build move enumeration.
15. **README inconsistencies.** The title says "Scenario 21" while the folder tree says `scenario-09-checkers` . `requirements.txt` is listed but wasn't uploaded. The submission checklist appears twice.

Not bugs: the initial board layout, promotion rows (R at row 0, B at row 7), turn alternation, and input validation for wrong count, non-numeric or out-of-range values all look correct. R moving first is a mismatch with the real rules, where the dark piece starts, but the README doesn't require it, so I'd leave it.

Where I'm least sure: I didn't execute anything, so bug 11 (alignment) and bug 2 (own-king jump) are from reading. You should reproduce both yourself, which the README wants for the "before" video anyway.

Which do you want first? Since it's a submission, I'd suggest working in task order, starting with Task 1's reproduction steps for your "before" video. Tell me when you want me to start writing code.

plan your approach to fixing the bugs from Tasks 1-4, it should be done with the least change in code as possible



Here's the plan. No code yet, and it deliberately reuses your existing `simple_move` and `capture_move` instead of rewriting them.

Assumptions

- American rules, as before: men move and jump forward only, kings move and jump both ways one square at a time, and a piece crowned mid-jump ends its turn.
- "Least change" means I don't restructure `run()` into smaller methods, even though it will get long. That's a real tradeoff. It keeps the diff small, but `run()` becomes harder to read.
- Line numbers below refer to your files as uploaded, and they'll shift as we edit.

The approach

1. `board.py` , `move_piece` (lines 17–19): removes the jumped piece (fixes bug 1) Add two lines. If the move spans two rows, clear the midpoint square. Capture removal then lives in one place, and since the piece disappears immediately, "can't be jumped twice" (bug 9) is fixed with no extra tracking.

2. `rules.py` : two small edits to existing functions (bugs 2 and 5)

- **Kings:** in `simple_move` (line 7) and `capture_move` (line 18), the fixed `direction` becomes "both directions if the piece at `start` ends with `K` , otherwise the one forward direction". The `er - sr == direction` checks become "is in allowed directions".
- **Own king:** in `capture_move` (line 24), compare only the first letter of the middle piece to `player` , so `"RK"` counts as your own piece.

3. `rules.py` : two new helper functions, about 10 lines each (bug 4)

- One reports whether a given piece has any capture available.
- One reports whether a player has any legal move at all.

Both loop over the squares and the four diagonal targets, calling your existing `capture_move` and `simple_move` . Nothing is duplicated, and these serve Tasks 2 and 3.

4. `game.py` , the acceptance block (lines 37–47): forced capture, multi-capture, promotion ordering (bugs 6, 7, 8)

- **Forced capture:** if the player has any capture available and the entered move isn't a capture, reject it with a message.
- **Multi-capture:** add one attribute, `self.chain_piece`, holding the landing square after a jump. While it is set, only that piece may be chosen, and only captures are allowed. If no further jump exists, clear it and switch players.
- **Promotion:** the chain ends immediately if the piece was crowned on this jump, even if it could keep jumping.

5. **game.py**, top of the loop (line 18): **win detection (bug 3)** Before printing the board each turn, check whether the current player has no pieces or no legal moves. If so, print the winner and return. The check uses helper 3.

6. **game.py** : **feedback (bug 10)** There will be one message in one place, immediately after a move is accepted, built from what just happened ("R moved", "R captured", "R promoted", "R must keep jumping"). Rejections keep their own messages. That satisfies "one accepted move, one result".

7. Small extras, worth doing but beyond the four tasks

- **Alignment (bug 11):** in `print_board` (line 13), pad each cell to width 2. This is a one-line change.
- **Ctrl-D and quit (bug 12):** wrap the `input` call (line 19) to exit cleanly. The README explicitly requires testing quitting, so I'd do this one.
- I'd skip the duplicate `SIZE` (bug 13) and the README typos (bug 15), since changing them adds diff without fixing behaviour.

Order of work

1. Reproduce the Task 1 bug and record your "before" video.
2. Apply steps 1 and 2 (capture removal, king rules) and test.
3. Apply step 3 (helpers), then step 4 (forced and multi-capture), then step 5 (win detection), then step 6 (feedback), then step 7.

The estimated total is about 40–50 changed lines across three files, with `main.py` untouched.

Where I expect trouble: the interaction of forced capture, chain continuation and mid-chain promotion is where these fixes usually break. Also, the "no legal moves" check must count captures, or a player with only a jump available would be declared stuck. I'll test both explicitly.

When we get to editing, I'll go one step (or a couple of tiny ones) per response, and I'll ask whether you want to paste the changes yourself or have me produce the files. Tell me when to start coding, and whether to begin with the Task 1 reproduction.



Here is the code for Task 1. I'm showing it inline, not writing any files, because your preference says to ask first. See the question at the end.

Assumption: I'm counting the own-king jump (bug 2) as part of Task 1, since it is also a capture-correctness defect. It's a one-line change. I've left king movement for Task 3, as planned.

Change 1: `board.py` , insert after line 19 (the end of `move_piece`)

Lines 17–19 stay exactly as they are. Add these two lines directly below, at the same 4-space indent as line 19:

python

```
if abs(end[0] - start[0]) == 2:
    board[(start[0] + end[0]) // 2][(start[1] + end[1]) // 2] = "."
```

After the change, the whole function reads:

python

```
def move_piece(board, start, end):
    board[end[0]][end[1]] = board[start[0]][start[1]]
    board[start[0]][start[1]] = "."
    if abs(end[0] - start[0]) == 2:
        board[(start[0] + end[0]) // 2][(start[1] + end[1]) // 2] = "."
```

This clears the jumped square whenever the move spans two rows. `game.py` only calls `move_piece` after `simple_move` or `capture_move` has accepted the move. So a two-row move can only be a validated capture, and the mover and destination are untouched.

Change 2: `rules.py` , replace line 24 (in `capture_move`)

Old:

python

```
board[mr][mc] not in (".", player)
```

New:

python

```
board[mr][mc] not in (".", player, player + "K")
```

This stops a piece from jumping your own king ("RK" or "BK").

How to reproduce and test

The reproduction is the same for your "before" video. Run these moves in order:

1. R: 5 2 4 3
2. B: 2 5 3 4
3. R: 4 3 2 5 (R jumps the B piece at row 3, column 4)

- **Before the fix:** after move 3, the B at (3,4) is still on the board. That is the bug.
- **After the fix:** (3,4) shows . , and R sits at (2,5).

I traced this by hand against the code and haven't executed it, so run it yourself to confirm. For the "cannot be captured again" half of the Done criterion, keep playing and check that no jump over that empty square is accepted. Your own-king fix can't be tested through normal play yet, since kings only appear after a promotion. I'd check it once Task 3 is in.

Question: do you want to paste these two changes into your local files yourself, or should I make the edited `board.py` and `rules.py` available for you to download? Reply "dump it into a file" if you want the latter, and I'll do that.

2026-09-30 11:28:16 You 

done, task 1 is done, move to task 2



Here is Task 2, in three small steps that all belong to the same feature.

Assumptions

- One check covers both end conditions. A player with no pieces also has no legal moves, so a single "has any legal move?" test handles both, with one message.
- The helper reuses your `simple_move` and `capture_move` . When we change those for kings in Task 3, the win check will pick up king moves automatically.
- I'm assuming you're pasting these yourself again. Say "dump it into a file" if you'd rather I generate the files.

Step 1: `rules.py` , add at the end of the file (after line 33)

Leave two blank lines after `promote` , then add:

python

```
def has_legal_move(board, player):
    for r in range(SIZE):
        for c in range(SIZE):
            if board[r][c] in (player, player + "K"):
                for dr in (-2, -1, 1, 2):
                    for dc in (-2, -1, 1, 2):
                        er, ec = r + dr, c + dc
                        if 0 <= er < SIZE and 0 <= ec < SIZE:
                            if simple_move(board, player, (r, c), (er, ec)) or \
                                capture_move(board, player, (r, c), (er, ec)):
                                return True

    return False
```

The bounds check is necessary. Without it, a negative index like `board[-1]` silently wraps to the last row in Python instead of raising an error, which would give false "legal moves".

Step 2: `game.py` , line 2

Old:

python

```
from rules import simple_move, capture_move, promote
```

New:

python

```
from rules import simple_move, capture_move, promote, has_legal_move
```

Step 3: `game.py` , insert after line 18 (`self.print_board()`), at the same 12-space indent

python

```
    if not has_legal_move(self.board, self.player):
        winner = "B" if self.player == "R" else "R"
        print(f"{self.player} has no pieces or no legal moves. {winner} wins!")
        return
```

I put it after `print_board()` so the final position is shown before the game ends. `return` exits `run()`, and `main.py` then finishes normally.

Testing

I traced these by hand and haven't run them. Use a Python shell in the project folder (`python`), so you don't have to play a full game:

1. No pieces:

`python`

```
from game import Checkers
g = Checkers()
g.board = [["."] * 8 for _ in range(8)]
g.board[0][1] = "B"
g.run()
```

Expected: the board prints, then "R has no pieces or no legal moves. B wins!", and the program exits.

2. Blocked piece (no legal moves but pieces remain):

same as above, but also set

`g.board[0][3] = "R"` before `g.run()`. An R man on row 0 has nowhere to go, since men only move up. Expected: the same message.

3. No false positive:

run `python main.py` normally. The game should start and accept moves, and `q` should still quit.

The one case I couldn't hand-test is a player whose only legal move is a capture, so please try it. Set up an R at (4,3), a B at (3,2), and (2,1) empty, with (3,4) and (2,5) both B so no simple move exists. It should not end the game.

Tell me what you see, and then we move to Task 3.

2026-09-30 11:36:51 You 

It works, move to Task 3



Task 3 has four parts, and I'm splitting it into steps so each one can be tested on its own. This response covers **king movement in both directions**. Forced capture, multi-capture and promotion timing come next.

Assumptions

- Kings move and jump one square at a time, in all four diagonal directions. Men stay forward-only, for moves and jumps.
- The direction check now depends on the piece standing on the start square, so it ends with `K` for kings. That is why `has_legal_move` from Task 2 picks up king moves automatically.
- **Promotion needs no change.** `promote()` already crowns R on row 0 and B on row 7, and those are the far rows, since R moves up and B moves down. I'll confirm that in the tests below.
- I'm assuming you're pasting these yourself.

Step 1: `rules.py` , `simple_move`

Replace line 7.

Old:

python

```
direction = -1 if player == "R" else 1
```

New:

python

```
directions = (-1, 1) if board[sr][sc].endswith("K") else ((-1,) if player == "R" else (1,))
```

Replace line 11.

Old:

python

```
er - sr == direction
```

New:

python

```
er - sr in directions
```

Step 2: `rules.py` , `capture_move`

Replace line 18 with the same line as in Step 1:

python

```
directions = (-1, 1) if board[sr][sc].endswith("K") else ((-1,) if player == "R" else (1,))
```

Replace line 23.

Old:

```
python
```

```
er - sr == 2 * direction and
```

New:

```
python
```

```
(er - sr) // 2 in directions and
```

The `abs(er - sr) == 2` check on the line above runs first, so `(er - sr) // 2` is always exactly -1 or 1 here.

Tests

I traced these by hand and haven't run them. Use a Python shell in the project folder:

```
python
```

```
from game import Checkers
g = Checkers()
g.board = [["."] * 8 for _ in range(8)]
g.board[0][1] = "B"           # keeps B alive so the game doesn't end
g.board[3][3] = "RK"
g.board[4][4] = "B"
g.run()
```

1. `3 3 2 2` should be accepted, a king moving forward. Then quit with `q`.
2. Rerun the setup, then `3 3 5 5` should capture the B at (4,4), a king jumping backward.
3. Rerun the setup but with `"R"` at (3,3) instead of `"RK"`. `3 3 5 5` should give "Invalid move.", since a man can't jump backward.
4. **Promotion:** set `g.board[1][2] = "R"` with B at (0,1) and nothing else for R, then play `1 2 0 3`. The piece should become `RK`. It will look misaligned, and that is bug 11, fixed later. I haven't tested B promotion on row 7 in the shell, so try a mirror setup with B at (6,1), R at (7,0) blocked by nothing, playing `6 1 7 2`.

Let me know what you see. Then the next response will cover forced capture and multi-capture, the part I flagged as the riskiest, so I'll keep those steps small.

2026-09-30 11:46:21 You 

give the rest of task 3 in 1 response

This response covers forced capture, multi-capture, and promotion ending the turn. Three files change: `rules.py` , then `game.py` in three places.

Assumptions

- The forced-capture rule applies to every piece, so if any of your pieces can jump, a plain move is rejected. You still choose which capture to make.
- A chain must continue with the same piece and must stop if the piece was just crowned. That is the American rule I verified earlier.
- The chain is tracked with one new attribute, `self.chain` , holding the landing square while a chain is in progress, or `None` otherwise.
- There is still no on-screen message when a chain continues, because feedback is Task 4. You'll see the same player's prompt again.
- Line numbers refer to the files as they are after your Task 2 edits. Do the `game.py` edits in the order below, since the first one changes the numbering below it.

Step 1: `rules.py` , add at the end of the file (after `has_legal_move`)

python

```
def has_capture(board, player, only=None):
    for r in range(SIZE):
        for c in range(SIZE):
            if (only is None or only == (r, c)) and board[r][c] in (player, player + "K"):
                for dr in (-2, 2):
                    for dc in (-2, 2):
                        er, ec = r + dr, c + dc
                        if 0 <= er < SIZE and 0 <= ec < SIZE and \
                            capture_move(board, player, (r, c), (er, ec)):
                            return True
    return False
```

`only` restricts the check to one square, which is used for chains.

Step 2: `game.py` , replace lines 41–51

Replace everything from `start, end = (sr, sc), (er, ec)` through `self.player = "B" if self.player == "R" else "R"` with:

python

```

start, end = (sr, sc), (er, ec)
if self.chain and start != self.chain:
    print("You must keep jumping with the same piece.")
    continue
was_man = not self.board[sr][sc].endswith("K")
if capture_move(self.board, self.player, start, end):
    jumped = True
elif simple_move(self.board, self.player, start, end):
    if has_capture(self.board, self.player):
        print("You must capture.")
        continue
    jumped = False
else:
    print("Invalid move.")
    continue

move_piece(self.board, start, end)
promote(self.board)
crowned = was_man and self.board[er][ec].endswith("K")
if jumped and not crowned and has_capture(self.board, self.player, end):
    self.chain = end
    continue
self.chain = None
self.player = "B" if self.player == "R" else "R"

```

The `elif simple_move` branch is where forced capture is enforced. It also blocks a plain move by the chain piece, because a capture is then available.

Step 3: `game.py` , insert after line 8 (`self.player = "R"`), at 8-space indent

python

```
self.chain = None
```

Step 4: `game.py` , replace line 2

python

```
from rules import simple_move, capture_move, promote, has_legal_move, has_capture
```

Tests

I traced these by hand and haven't run them. Use the shell setup pattern from before, starting each with:

python

```
from game import Checkers
g = Checkers()
g.board = [["."] * 8 for _ in range(8)]
g.board[0][1] = "B"
```

1. **Forced capture:** add `g.board[4][3] = "R"` , `g.board[3][4] = "B"` , `g.board[6][1] = "R"` , then `g.run()` .

- `6 1 5 0` should print "You must capture."
- `4 3 2 5` should be accepted.

2. **Multi-capture:** add `g.board[6][1] = "R"` , `g.board[5][2] = "B"` , `g.board[3][4] = "B"` , `g.board[7][0] = "R"` .

- `6 1 4 3` should capture (5,2), and the prompt should still say `R>` .
- `7 0 6 1` should print "You must keep jumping with the same piece."
- `4 3 3 2` should print "You must capture."
- `4 3 2 5` should capture (3,4), and then it's B's turn.

3. **Promotion ends the turn:** add `g.board[2][1] = "R"` , `g.board[1][2] = "B"` , `g.board[1][4] = "B"` .

- `2 1 0 3` should crown the piece to `RK` and pass the turn to B (`B>`), even though the new king could jump (1,4). The display will look misaligned, which is bug 11 and gets fixed with Task 4.

Most likely to fail: the mid-chain promotion in test 3 and the "same piece only" rule in test 2. Both come from the interaction between the new `chain` state and the win check at the top of the loop. Report anything unexpected and I'll trace it.

2026-09-30 11:51:11 You 

everything works, move to task 4



Task 4 is feedback. I've also included the two small extras from my plan (board alignment and clean quitting), because the README's required tests cover quitting and the misaligned king display makes the feedback look sloppy. They're marked optional.

Assumptions

- Each accepted input is one "move", so each jump in a multi-capture chain prints its own line. Rejected inputs keep their existing error messages and print no action line.

- Exactly one action line is printed per accepted move, built in one place from what just happened. That covers "even if several internal legal-move checks occur", because `has_capture` and `has_legal_move` print nothing.
- Line numbers are approximate after your earlier edits, so identify each spot by its content.

Step 1: `game.py` , in `run()` , replace the tail of the move handling

Find these lines, just after `promote(self.board)` :

Old:

python

```
crowned = was_man and self.board[er][ec].endswith("K")
if jumped and not crowned and has_capture(self.board, self.player, end):
    self.chain = end
    continue
self.chain = None
```

New:

python

```
crowned = was_man and self.board[er][ec].endswith("K")
again = jumped and not crowned and has_capture(self.board, self.player, end)
msg = f"{self.player} {'captured' if jumped else 'moved'} {start} -> {end}"
if crowned:
    msg += " and was crowned king"
print(msg + (" . Jump again!" if again else "."))
if again:
    self.chain = end
    continue
self.chain = None
```

The `self.player = "B" if ...` line below stays as it is.

Step 2 (optional): `game.py` , `print_board` , fix the alignment

Two lines, about lines 12 and 14 (find by content).

Old header:

python

```
print("\n  " + " ".join(str(c) for c in range(SIZE)))
```

New:

python

```
print("\n    " + " ".join(str(c).ljust(2) for c in range(SIZE)))
```

Old row:

```
python
```

```
print(f"{r}    " + " ".join(row))
```

New:

```
python
```

```
print(f"{r}    " + " ".join(cell.ljust(2) for cell in row))
```

Step 3 (optional): game.py , run() , clean quitting

Replace the `raw = input(...)` line (about line 24) with:

```
python
```

```
try:
    raw = input(f"{self.player}> ").strip().lower().split()
except (EOFError, KeyboardInterrupt):
    print("\nGoodbye.")
    return
```

This makes Ctrl-D and Ctrl-C exit cleanly instead of showing a traceback. The existing `q` still works.

Tests

I traced these by hand and haven't run them. With the shell setup from before (`g.board = [["."] * 8 for _ in range(8)]` , B at (0,1) to keep the game alive):

1. **Plain move:** R at (5,2), then `5 2 4 3` prints R moved (5, 2) -> (4, 3).
2. **Capture:** R at (4,3), B at (3,4), then `4 3 2 5` prints R captured (4, 3) -> (2, 5).
3. **Chain:** use the multi-capture setup from Task 3. The first jump prints R captured (6, 1) -> (4, 3). Jump again! , and the second prints R captured (4, 3) -> (2, 5). , with no "Jump again!".
4. **Promotion:** R at (1,2), then `1 2 0 3` prints R moved (1, 2) -> (0, 3) and was crowned king. After Step 2, the board should stay aligned with RK on it.
5. **Errors:** `9 9 9 9` gives "Outside board." and no action line. `abc` gives "Enter four coordinates." A non-capture move while a capture is available gives "You must capture." (no action line).
6. **Quitting:** `q` exits, and Ctrl-D exits with "Goodbye." and no traceback.

One thing to check: a promotion that happens with a capture reads "captured ... and was crowned king". That is intended, and it counts as one message for one move.

Once these pass, all four tasks are done. For the README's required tests and your after-video, I can write a checklist that covers every required case (ordinary moves through quitting) in one gameplay script, if you want it.